COMPASS: A Chain-of-Thought Approach toward Geo-Spatial Reasoning for Popular Path Query using LLMs

Author information scrubbed for double-blind reviewing

No Institute Given

Abstract. Geo-spatial reasoning problems, such as identifying popular paths from historical trajectory data, are challenging due to the complexity and limitations of traditional algorithms and machine learning methods. These approaches often fail when synthesizing novel paths under user-defined constraints or sparse data. We introduce **COMPASS**, a novel framework that intelligently leverages the reasoning capabilities of Large Language Models (LLMs) for complex geo-spatial tasks. **COMPASS** employs a two-stage approach: a "Search" stage that identifies popular paths, and a "Generate" stage that synthesizes new paths, both harnessing LLMs' ability to understand and reason about spatial relationships, constraints, and graph structures from historical data. Extensive experiments on real and synthetic datasets show that **COMPASS** not only performs well in standard comparisons, it excels where traditional methods fail, especially in generating novel paths and responding with user defined constraints. We will open-source the implementation of **COMPASS**.

1 Introduction

Mapping the most frequented routes between key locations from historical trajectory (i.e., previous travelled paths) data—referred to as popular path discovery and synthesis—unlocks valuable insights for urban planning, navigation optimization, robotics, autonomous vehicle operations, and personalized travel or route recommendations [44, 17, 41, 42].

Developing an effective solution for popular path queries is fundamentally complex, with challenges that are multi-fold. At its core, the problem is framed as identifying the most weighted path in a graph, where edges represent historical trajectories and weights reflect their frequency; this formulation is NP-hard [7, 24]. While heuristic and approximation algorithms exist [5, 19, 34], they struggle with large-scale datasets and complex constraints. Incorporating factors like path length and user-defined node inclusions or exclusions further complicates the task. As problem complexity increases, conventional algorithmic approaches become increasingly difficult to implement, debug, and scale effectively, highlighting the need for more flexible and robust solutions.

These challenges further encompass the accurate interpretation of large-scale trajectory data and the efficient synthesis of paths, especially when no direct path exists within the given trajectories (See Fig 1). The necessity for scalability, real-time responsiveness, and the capability to manage noisy or incomplete data compounds these issues. Tackling

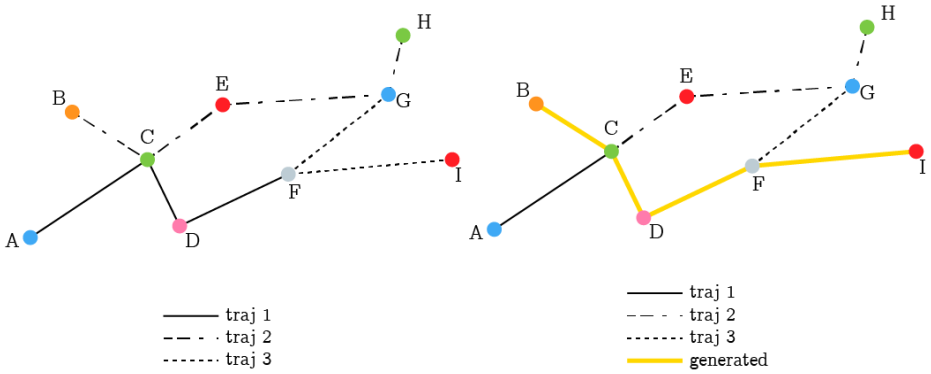


Fig. 1: COMPASS *workflow example: Given three trajectories (**BCEGH**, **ACDF**, and **GFI**),* **COMPASS** *is asked to find the most popular path from **B** to **I**. First, it searches the existing trajectories (left), but since none span the entire route, it enters the generate phase (right). In this phase,* **COMPASS** *breaks down the trajectories into sub-trajectories: **BC**, **CDF**, and **FI**, which are combined to form the new path **BCDFI** (in yellow).*

these complexities is crucial for developing robust solutions that can function effectively across diverse real-world scenarios.

To address this, earlier machine learning approaches for popular path queries were limited to existing paths in historical data and required long-training [37, 30]. More recent neural network models like NeuroMLR [10] can generate new paths using Graph Neural Networks, but need complete map data. While effective for popular path synthesis, NeuroMLR requires retraining to accommodate user constraints or data updates, making it impractical for scenarios needing frequent adjustments. Thus, developing a flexible AI system for identifying and synthesizing popular paths from historical trajectory data remains a key challenge in geo-spatial reasoning.

Meanwhile, Large Language Models (LLMs) have excelled in planning tasks, demonstrating proficiency in processing long-context input for global insights [14]. Their zero-shot & in-context inference capabilities enable adaptation to new domains without system modifications. Framing geo-spatial problems as natural language prompts enables LLMs to generate solutions addressing complex spatial relationships & constraints, potentially overcoming traditional challenges of inflexibility & computational complexity.

The Chain-of-Thought (CoT; [33]) prompting paradigm has emerged as a potent tool, offering a single-shot response that incorporates underlying explanations to enhance the reasoning capabilities of LLMs. While CoT and its subsequent methods, including Self-Consistency (SC; [31]), ReACT [39], and Reflexion [26], generalize CoT with various multi-objective, ensemble-based, or tool-augmented, and trial-and-error approaches, they often fall short in geo-spatial reasoning—particularly in path synthesis—where LLMs tend to generate invalid or ungrounded paths that are incomplete or collide, revealing limitations in handling complex spatial constraints [1]. Most recently, LLM-A* [20] combines the A* algorithm with LLMs, achieving enhanced path synthesis validity but neglecting popular path queries. Furthermore, these iterative variants violate the

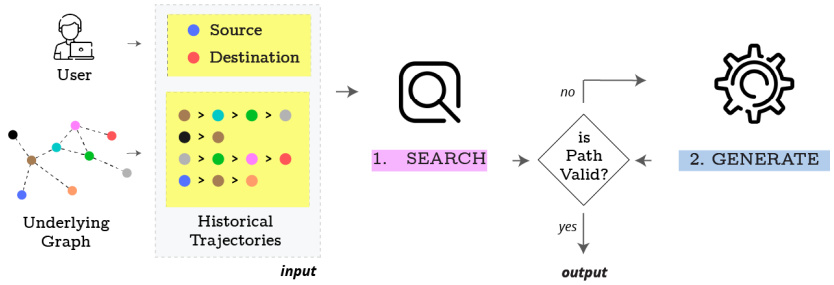


Fig. 2: *Overview of* **COMPASS** *- A two-stage framework for optimal path search and generation in complex networks. It takes a list of historical trajectories and a query specifying a source destination pair as input. The underlying graph is not known to* **COMPASS**. *The Stage 1 (**SEARCH**) finds the best path considering popularity of POIs when a direct path exists. Stage 2 (**GENERATE**) creates a popular path when no direct route is available, leveraging historical trajectory data to synthesize new paths.*

real-time requirements essential for this task. Thus, unlocking CoT's true potential for geo-spatial reasoning in popular path queries remains an unresolved challenge.

To address this, we introduce a simple, verification-free, zero-shot CoT-like prompting framework, **COMPASS** (COnstrained Multi-Purpose pAth Search and Synthesis), featuring a novel two-stage framework (*Search* and *Generate*) that leverages Large Language Models (LLMs) to tackle these challenges. The *Search* stage finds popular paths from historical data, while none exists, the *Generate* stage in **COMPASS** adaptively synthesizes new paths using graph structures derived from the data.

Devising a robust prompt for such a complex of popular path finding (*Search* stage) and synthesis (*Generate* stage) is non-trivial and decision making procedure in LLMs are opaque. Therefore, to design **COMPASS**, we take a data driven approach by fine-grained qualitative analysis and reverse engineer. We consider one city map and its trajectory dataset from previous works to perform the case-study. We first prompt the problems with basic CoT ("think step-by-step") instruction with varying temperatures and gather all the responses for all the problems within. We manually analyze the explanation and the answer. We annotate a set of argument types such as path candidates, scores, the evaluation process and ranking in the explanations that are present in large in the correct responses for each of (*Search*) and (*Generate*) stages. Based on these, we manually articulate two different prompts asking explicitly to produce those arguments along with the answer (See Fig 2 for an overview).

In our evaluation, we utilize both real-world city maps from user movement tracking and synthetic datasets. While real data revealed natural "highway" patterns of frequent user movement, it lacked diverse spatial characteristics. To address this, we generated synthetic data using a reverse delete algorithm to create connected graphs, and incorporated Steiner trees to simulate high-traffic paths. Users in our simulation followed these highways 90% of the time, deviating only for shorter alternatives. This comprehensive dataset enables thorough evaluation of COMPASS across diverse spatial configurations, demonstrating its effectiveness in novel path generation and constraint adaptation where

traditional methods struggle. Our evaluation shows the **COMPASS** consistently outperforms or matches all existing baselines while being real-time (non-iterative) i.e. requires no model training and also being cost-effective in terms of token usage.

2 Related Works

Route recommendation and prediction have been central research topics, driven by the increasing availability of trajectory data and the growing demand for personalized navigation services. Existing approaches can be broadly classified into three categories: algorithmic methods, machine learning models, and more recently, large language model (LLM)-based approaches.

2.1 Algorithmic Approaches

Algorithmic approaches for route recommendation are inherently complex. A naive approach may suggest that simply sorting and returning the most frequently visited path would yield the most popular route, but this is a misconception, as illustrated in Fig 3. Early research in this field predominantly employed probabilistic and heuristic methods [2, 5, 19, 34, 36]. However, these methods often require significant manual effort to handle corner cases, and when new constraints are introduced, they demand additional manual adjustments. This reliance on manual coding becomes impractical in today's fast-paced world, where adaptability and scalability are paramount.

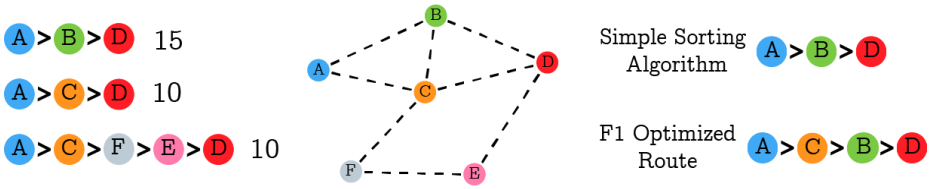


Fig. 3: *The map shows three historical routes: the first (**ABD**) is taken **15** times, and the other two (**ACD**, **ACFED**) are taken **10** times each. A basic algorithm might select the **most frequently** traveled route (**ABD**), but this would miss important **POI** like the **C**, which is highly visited across multiple routes. Simple **frequency-based sorting** is insufficient for identifying the most **popular** path.*

2.2 Machine Learning Approach

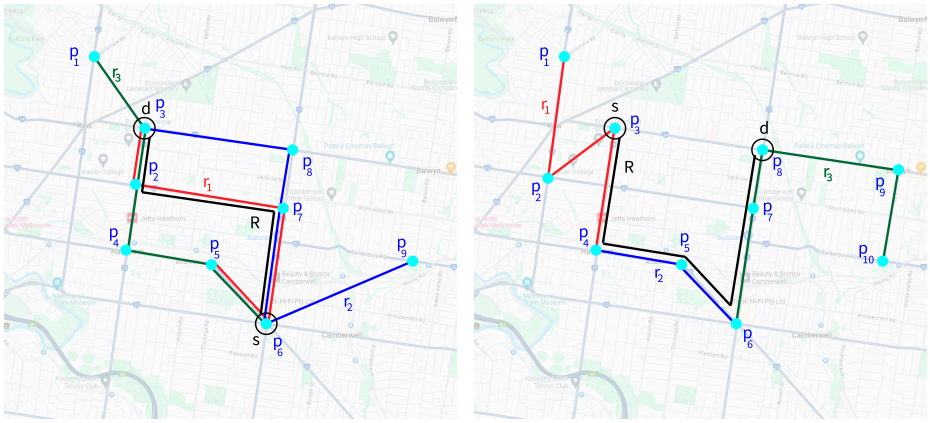
In recent years, machine learning has become the primary focus in route recommendation techniques [35, 15, 43]. While these approaches are generally more robust, they initially lacked generative capabilities. Deep generative models [10, 35] have since addressed this limitation. However, machine learning models remain inherently dependent on the data they are trained on, requiring time-consuming retraining when data changes. In scenarios

like route recommendation, where data may change frequently due to environmental, natural, or economic factors, retraining becomes a significant bottleneck.

2.3 LLM-based Approaches

Large Language Models (LLMs) have emerged as a revolutionary tool, applied across various domains, from education to software development. Techniques such as prompt engineering [32, 31, 38, 39, 26], fine-tuning [9], and prompt-tuning have opened new avenues for LLMs to be applied in diverse fields. In particular, LLMs have demonstrated potential in areas such as graph reasoning [6, 40, 29] and task planning [27, 8, 25]. Nevertheless, challenges remain in ensuring that LLM-generated routes conform to real-world road networks [12]. Key issues include the inherent inconsistency of LLM outputs [11] and difficulties in parsing complex input structures [28]. Despite these obstacles, the potential for LLMs to enhance the quality of route recommendations and overall user experience is considerable, indicating a promising direction for future research.

3 COMPASS



(a) *Search Problem* **(b)** *Generate Problem*

Fig. 4: ***(a)****Based on three historical routes R_1 (red), R_2 (blue), & R_3 (green) and for a query with source-destination pair $< s, d >=< p_3, p_6 >$, as paths do exist between p_3 and p_6, the LLM SEARCHes and finds a path R (black).* ***(b)****Based on three historical routes R_1 (red), R_2 (blue), & R_3 (green) and for a query with source-destination pair $< s, d >=< p_3, p_8 >$, as no direct path between p_3 and p_8 exists, the LLM GENERATEs a path R (black)*

Our proposed solution, **COMPASS**, addresses the challenge of geo-spatial reasoning using large language models (LLMs) to solve the Popular Path Problem. This section

outlines the design and implementation of **COMPASS**, including problem definition, the two-phase framework, and the prompt engineering strategies employed to achieve optimal path generation. The entire pseudo-code algorithm can be found in [Appendix A]

3.1 Problem Definition

Let $\mathcal{V} = p_1, p_2, \ldots, p_N$ be a set of N POIs in a city. The historical trajectories, denoted as $\mathcal{T} = r_1, r_2, \ldots, r_M$, represent a collection of M routes that people have taken while visiting the city in the past. Each route $r_i \in \mathcal{T}$ is an ordered sequence of POIs i.e. $r_i = (p_{i1}, p_{i2}, \ldots, p_{iJ})$, where J is the number of POIs along that particular route. Given these historical trajectories $\mathcal{T}$ and a query $< s, d >$, where s is the starting POI and d is the destination POI, our aim is to find the single most popular path $\mathcal{R} = (q_1, q_2, \ldots, q_K)$. This path should satisfy the conditions that $q_1 = s$, $q_K = d$, and $\forall k, q_k \in \mathcal{V}$. The path $\mathcal{R}$ is an ordered sequence of K POIs, representing a simple path in the graph where $\mathcal{V}$ is the set of vertices & $\mathbf{E}$ is the set of edges. Here, $\mathbf{E} = \{(p_j, p_{j+1}) \mid \exists\, r = (p_1, p_2, \ldots, p_J) \in \mathcal{T}$ such that $1 \leq j \leq J\}$

The challenge lies in determining what constitutes the most **popular** path, especially in cases where no single historical route exactly matches the query's start and end points. This leads us to consider two distinct scenarios: *Search* and *Generate*.

3.2 COMPASS Framework

COMPASS operates in two phases, each comprising multiple steps. Figure 2 shows an overview of the entire pipeline, and Appendix D provides the exact prompts and responses.

SEARCH Phase

*S*1*: POI Popularity Calculation.* To identify popular POIs, we define a *popularity score map* $\mathbf{P} : \mathcal{V} \to \mathbb{R}_{\geq 0}$, where $\mathbf{P}(p_i)$ measures how frequently POI $p_i \in \mathcal{V}$ appears in all trajectories $\mathcal{T}$. Higher values indicate more popular POIs. This step helps establish a quantitative measure of popularity, ensuring that frequently visited locations have higher influence in path selection.

*S*2*: POI Popularity Ranking.* Rank all POIs by descending $\mathbf{P}(p_i)$. Let $\mathrm{rank}(p_i)$ denote the position of p_i in this sorted list, with $\mathrm{rank}(p_i) = 1$ indicating the most frequently visited POI. This ranking allows us to prioritize highly visited locations in subsequent path evaluations, ensuring that frequently chosen POIs contribute more significantly to route formation.

*S*3*: Path Identification.* From $\mathcal{T}$, identify routes and sub-routes $(p_j, \ldots, p_{j+k})$ where $p_j = s$ and $p_{j+k} = d$. These sub-routes inherit the POI popularity context from $\mathbf{P}$. By extracting these paths, we ensure that we only consider historically traversed routes, allowing us to focus on empirically validated trajectories rather than hypothetical ones.

S4: Path Availability Check. If no route from s to d is found, proceed to the *Generate* phase. Otherwise, continue to extract all candidate paths. This step ensures that if historical data does not provide a valid path, we attempt to synthesize new ones, making the framework robust to data sparsity.

S5: Extract All Paths. Collect the paths from $S3$. These paths are the candidates for being the popular path. Let $\mathbf{C}$ denote the resulting set of candidate paths:

$$\mathbf{C} = \{\, (p_j, \ldots, p_{j+k}) \;\mid\; \exists\, (p_1, \ldots, p_J) \in \mathcal{T} \text{and } p_j = s,\ p_{j+k} = d,\ 1 \le j,\ j{+}k \le J\}.$$

This step guarantees that all potential paths from historical data are gathered before evaluating them for popularity and efficiency.

S6: Evaluate Paths. Assess each path in $\mathbf{C}$ using criteria such as aggregate popularity (via $\mathbf{P}$) and path length. By considering these factors, we balance the trade-off between choosing highly popular routes and ensuring they remain efficient for traversal.

S7: Select Best Path. From $\mathbf{C}$, select the single most popular and efficient path, denoted by $\mathcal{R}$. Return $\mathcal{R} = (q_1, q_2, \ldots, q_K)$ as the recommended route.

GENERATE Phase

G1: Extract Edges. Define the set of edges $\mathbf{E}$ by scanning consecutive POIs in all $r_i \in \mathcal{T}$:

$$\mathbf{E} = \{(p_j, p_{j+1}) \mid \exists\, (p_1, p_2, \ldots, p_J) \in \mathcal{T} \text{ such that } 1 \le j \le J\}.$$

This step establishes connectivity between POIs, forming the basis for constructing alternative paths when historical data lacks direct trajectories.

G2: Edge Frequency Analysis. Let $\mathbf{F} : \mathbf{E} \to \mathbb{N}$ be a frequency function where $\mathbf{F}(e)$ is the count of how often edge $e \in \mathbf{E}$ appears across all $r_i \in \mathcal{T}$. High $\mathbf{F}(e)$ indicates a more frequently traveled connection. By tracking edge frequencies, we can prioritize well-traveled connections when generating paths.

G3: Generate Path Candidates. Construct new path candidates from s to d using the most frequent edges. Denote this set of synthesized paths by $\mathbf{G}$:

$$\mathbf{G} = \{\, (p_1, \ldots, p_J) \;\mid\; p_1 = s,\ p_J = d \text{ and } (p_j, p_{j+1}) \in \mathbf{E} \text{ for all } 1 \le j \le J\}.$$

G4: Score Paths. Assign a score to each path in $\mathbf{G}$ based on edge frequencies $\mathbf{F}(e)$, path length, or user-defined preferences. Let $\mathbf{S} :\ \mathbf{G} \to \mathbb{R}_{\ge 0}$ represent the mapping from each candidate path to its score. This scoring function allows us to objectively compare synthesized paths, ensuring that the best candidates are prioritized.

G5: Path Selection Reasoning. Select the candidate path $g^* \in \mathbf{G}$ with the highest score $\mathbf{S}(g^*)$. Provide a brief rationale that justifies why g^* is optimal. Finally, output g^* as the final recommended path, denoted by $\mathcal{R} = (q_1, q_2, \ldots, q_K)$ where $q_1 = s$ and $q_K = d$.

4 Experiments

4.1 Dataset

To thoroughly evaluate the performance of **COMPASS**, we tested it on both real-world and synthetic datasets to ensure its robustness and versatility across different contexts.

Real-World Data We used seven real-world datasets across two domains: geo-tagged Flickr traces from Edinburgh (Edin), Toronto (Toro), and Melbourne (Melb) [3], and trip data from four theme parks, Disney's Hollywood Studios (DisHolly), Epcot Theme Park (Epcot), Disney California Adventure (DisCally) & Disneyland (Disland) [18]. Each trajectory was cleaned by removing duplicate POIs and restricting the time gap between consecutive points to 8 hours, ensuring meaningful path analysis. While all the datasets were used for SEARCH problem, only Edin, Toronto & Melbourne were used for the GENERATE problem as other datasets didn't have any POI pair that doesn't have any historical trajectory and could serve as source-destination.

Synthetic Data We generated synthetic datasets to introduce more balanced edge usage and spatial diversity, addressing the skewed distribution in real data. Using the reverse-delete algorithm, we controlled edge density while a Steiner tree approximation [23] emphasized key "highway" routes. Users followed these highways 90% of the time, deviating only when shorter paths were available, closely mirroring real-world movement patterns.

The data we generated through this follows the distribution of Real data closely. Fig 7 in the Appendix shows the edge distribution of a real data and a synthetic data that is given similar parameter (13 POI and 901 Trajectory). Appendix B provides the entire algorithm for synthetic data generation in details.

4.2 Comparison

We evaluated **COMPASS** against traditional machine learning, deep learning, and LLM-based models. The **Markov** model [4] predicts user movements with Markov chains, while **NASR** [30] employs neural architecture search for urban navigation. **DeepAltTrip** [22] generates diverse alternative routes, and **NeuroMLR** [10] creates paths for large-scale traffic networks using graph-based learning. Among LLM techniques, **CoT** [32] breaks tasks into sequential steps, **Self-Consistency** [31] selects the most consistent reasoning paths, and **APE** [45] automates task-specific prompt generation. **ReAct** [39] integrates reasoning with action, **Reflexion** [26] enhances performance through iterative feedback, and **LLM A*** [21] adapts A* search for improved spatial reasoning. For LLM based method we used both an open source (Llama 3.1 8B) and an close source model (GPT4o)

4.3 Models & Environment

For each method, prompts were crafted following the respective guidelines of the technique. We recorded the number of tokens used in the prompt as well as the tokens

Category	DisHolly 13 POIs	Epcot 17 POIs	CaliAdv 25 POIs	Edin 28 POIs	Toro 29 POIs	Disland 31 POIs	Melb 88 POIs	Average -
ML/DL Based								
Markov	0.61	0.59	0.53	0.59	0.60	0.50	0.51	0.56
NASR	0.63	0.60	0.54	0.58	0.58	0.50	0.53	0.57
DeepAltTrip-LSTM	0.62	0.59	0.53	0.58	0.59	0.52	0.52	0.56
DeepAltTrip-Samp	0.62	0.58	0.53	0.58	0.59	0.51	0.51	0.56
NeuroMLR	0.70	0.69	0.55	0.62	0.69	0.65	0.60	0.65
LLM Based (Llama 3.1 8b)								
Direct (Zero Shot)	0.50	0.46	0.47	0.45	0.45	0.43	0.40	0.45
CoT	0.59	0.53	0.56	0.54	0.57	0.48	0.47	0.53
Self-Consistency	0.57	0.55	0.54	0.56	0.54	0.50	0.45	0.53
APE	0.45	0.40	0.42	0.41	0.38	0.34	0.32	0.39
ReAct	0.68	0.64	0.66	0.70	0.59	0.47	0.43	0.60
Reflexion	0.69	0.64	0.66	0.70	0.59	0.48	0.44	0.60
LLM A*	0.58	0.53	0.55	0.60	0.52	0.42	0.40	0.51
COMPASS SEARCH	0.75	0.69	0.71	0.76	0.62	0.49	0.44	0.64
LLM Based (GPT 4o)								
Direct (Zero Shot)	0.56	0.51	0.52	0.50	0.50	0.48	0.45	0.50
CoT	0.66	0.57	0.61	0.59	0.62	0.51	0.52	0.58
Self-Consistency	0.64	0.59	0.58	0.61	0.59	0.53	0.49	0.58
APE	0.79	0.71	0.75	0.81	0.65	0.50	0.45	0.67
ReAct	0.74	0.70	0.73	0.74	0.64	0.49	0.45	0.64
Reflexion	0.74	0.70	0.73	0.74	0.64	0.51	0.46	0.65
LLM A*	0.61	0.56	0.57	0.62	0.54	0.45	0.42	0.54
COMPASS SEARCH	0.80	0.73	**0.74**	0.81	0.65	0.50	0.45	0.67
LLM Based (GPT o3 mini)								
Direct (Zero Shot)	0.74	0.73	0.59	0.57	0.68	0.67	0.65	0.66
CoT	0.82	0.73	0.71	0.75	0.84	0.67	0.78	0.76
Self Consistency	0.74	0.76	0.57	0.68	0.84	0.64	0.80	0.72
APE	0.81	0.74	0.71	0.80	0.89	0.64	**0.81**	0.77
ReAct	0.75	0.73	0.61	0.72	0.87	0.63	0.77	0.73
Reflexion	0.75	0.75	0.65	0.77	0.81	0.52	0.77	0.72
LLM A*	0.75	0.73	0.72	0.67	0.87	**0.72**	0.77	0.77
COMPASS SEARCH	**0.84**	**0.79**	0.73	**0.82**	**0.90**	0.71	**0.81**	**0.80**

Table 1: *F1 Score Comparison across locations. Higher F1 scores indicate better alignment with popular POIs. APE performs well with GPT 4o but struggles with Llama 3.1 8b due to model's limitation. ReAct and Reflexion perform well with many steps but falter with* **COMPASS***'s three-step limit.* **COMPASS** *consistently outperforms other methods with efficient prompting.*

generated in the response. The experiments were executed using GPT 4o, GPT o3 mini and Llama 3.1 8b as the underlying language model. The reason behind not picking other newer models is they often take very long to infer which is impractical in the real-time scenarios that **COMPASS** fixes. We maintained a lower *temperature* across all experiments as it shows better result [Appendix C.1]. Additionally, a medium *reasoning*

Category	**Edin** 28 POIs	**Toro** 29 POIs	**Melb** 88 POIs	**Average** -
ML/DL Based				
NeuroMLR	0.89	**0.98**	0.95	0.94
LLM Based (Llama 3.1 8b)				
Direct (Zero Shot)	0.12	0.38	0.59	0.36
CoT	0.21	0.23	0.19	0.21
Self-Consistency	0.19	0.22	0.21	0.21
APE	0.18	0.11	0.02	0.10
ReAct	0.69	0.79	0.81	0.76
Reflexion	0.71	0.77	0.78	0.75
LLM A*	0.78	0.82	0.49	0.70
COMPASS GENERATE	0.72	0.78	0.61	0.70
LLM Based (GPT 4o)				
Direct (Zero Shot)	0.30	0.21	0.33	0.28
CoT	0.48	0.60	0.20	0.43
Self-Consistency	0.50	0.60	0.22	0.44
APE	0.86	0.89	0.94	0.90
ReAct	0.82	0.85	0.90	0.86
Reflexion	0.83	0.86	0.91	0.87
LLM A*	0.84	0.82	0.90	0.85
COMPASS GENERATE	0.84	0.91	**0.96**	0.90
LLM Based (GPT o3 mini)				
Direct (Zero Shot)	0.54	0.51	0.53	0.53
CoT	0.56	0.51	0.56	0.54
Self-Consistency	0.83	0.56	0.53	0.64
APE	0.80	0.71	0.88	0.80
ReAct	0.56	0.51	0.63	0.57
Reflexion	0.54	0.63	0.91	0.69
LLM A*	0.56	0.53	0.90	0.66
COMPASS GENERATE	**0.95**	0.94	0.95	**0.95**

Table 2: *Reachability scores across locations. Only NeuroMLR is used among ML/DL methods, as others failed to generate paths without trajectories. Although* **COMPASS** *GENERATE does not surpass ML/DL methods, ML/DL does need retraining on new data, where* **COMPASS** *can be used instantly*

effort was used for the reasoning model, GPT o3 mini, so that it doesn't take too long to reason which would disrupt **COMPASS**'s purpose of real-time path recommendation.

4.4 Metrics

We used two key metrics: $F1Score$ for SEARCH problems and $Reachability$ for GENERATE problems.

The $F1Score$, a harmonic mean of precision and recall, evaluates the accuracy of the recommended paths by comparing them to the existing routes in the historical trajectories.

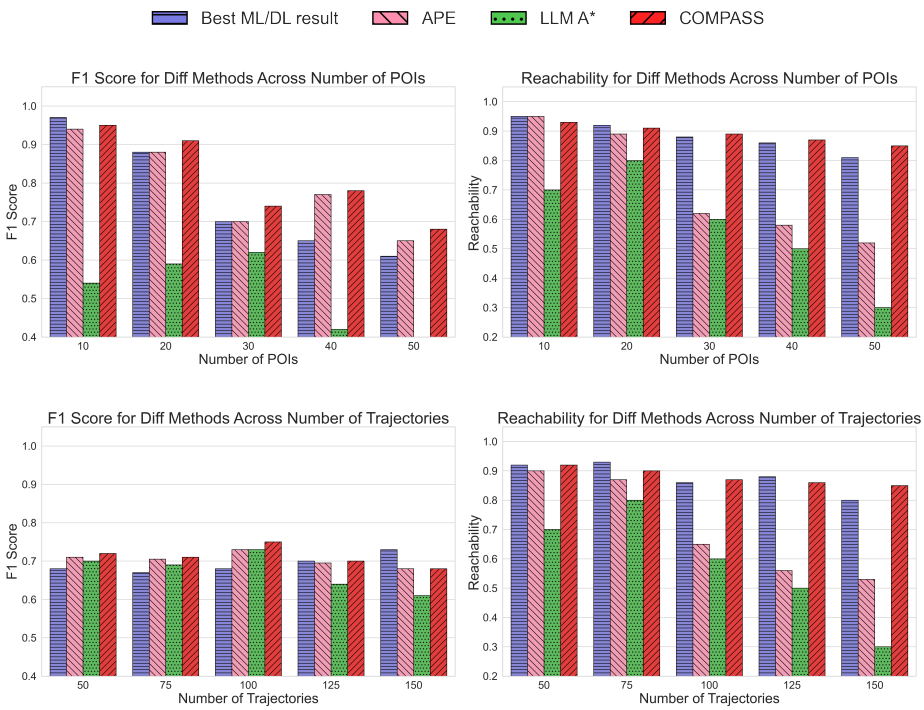


Fig. 5: *Synthetic Data Evaluation: F1 Score and Reachability comparisons across various methods as the number of Points of Interest (POIs) and trajectories increases. Synthetic data was used to simulate diverse spatial environments and evaluate the methods' performance under controlled conditions. The "Best ML/DL method" refers to the most competitive machine learning or deep learning model trained on trajectory data. A general downward trend is observed as the map size (POI) or input size (Trajectories) increases, with APE performing well but incurring high computational costs.* **COMPASS** *demonstrates consistent performance across different scenarios, especially in low-resource conditions. In reachability, ML/DL methods perform well but require significant training time, while* **COMPASS** *offers competitive reachability without the need for retraining.*

Let $r = (p_1, p_2, p_3, ..., p_{|r|})$ be a ground truth route where $p_1 = s$ and $p_{|r|} = d$, and let $\mathcal{R} = (q_1, q_2, ..., q_{|\mathcal{R}|})$ be a recommended itinerary. Also, denote Q_r and $Q_{\mathcal{R}}$ as the sets of Points of Interest (POIs) in the ground truth route and recommended itinerary, respectively. The precision(P), recall (R) and $F1Score$ of the recommended itinerary $\mathcal{R}$ is calculated as:

$$P = \frac{|Q_r \cap Q_{\mathcal{R}}|}{|Q_{\mathcal{R}}|} \qquad R = \frac{|Q_r \cap Q_{\mathcal{R}}|}{|Q_r|} \qquad F1 = \frac{2 * P * R}{P + R} \tag{1}$$

$Reachability$ measures the model's ability to generate paths that connect source and destination points using consecutive POIs from historical routes. Mathematically it is defined as,

$$\mathcal{R} = \{q_1, q_2, \ldots, q_K\} \text{ is } reachable \text{ if } \forall i, 1 \leq i < K, (q_i, q_{i+1}) \in \mathcal{V}$$

5 Result Analysis

5.1 Performance of COMPASS

The performance evaluation of **COMPASS** across multiple datasets, as presented in Table 1, 2, and Fig. 5 highlights its effectiveness in retrieving optimal paths. In fact, LLM based methods consistently outperform traditional ML/DL-based methods in the SEARCH phase. **COMPASS**, by efficiently leveraging historical data and the retrieving high-quality paths, gained a better F1 scores compared to standard approaches such as Markov models, NASR, and deep learning-based baselines and even other LLM based methods. As GPT o3 mini is a reasoning model, it has a overall better performance over GPT 4o and Llama 3.1 8b.

In the GENERATE phase, deep learning models like NeuroMLR achieve almost similar reachability scores as **COMPASS**, as seen in Table 2. However, this comes at the cost of requiring significant retraining on new data, limiting their adaptability in dynamic environments. In contrast, **COMPASS** maintains competitive reachability performance while offering a more practical and scalable solution. The ability of **COMPASS** to operate without extensive retraining makes it well-suited for real-time applications where rapid adaptation to new conditions is crucial.

Switching to synthetic data to analyze **COMPASS**'s behavior across different spatial configurations, we observe similar findings, as shown in Figure 5. APE and **COMPASS** are consistent with their performance as discussed further in Appendix C.3. **COMPASS**, however, remains robust even as the number of Points of Interest (POIs) and trajectories increases, ensuring reliable performance across diverse spatial environments.

5.2 Cost Analysis

A key advantage of **COMPASS** is its ability to balance performance with computational efficiency, as demonstrated in Table 3, keeping only the best performing methods. APE, while achieving strong results, incurs significantly higher computational costs due to its reliance on extensive prompt engineering. On the other hand, LLM-A* is the most cost-effective method but struggles with performance, making it unsuitable for high-quality path generation. Other methods, such as REACT and Reflexion, exhibit a quadratic increase in cost at each step, rendering them highly impractical for regular or fast usage unless constrained to a fixed number of steps.

COMPASS strikes an optimal balance by achieving competitive results with moderate computational overhead. For instance, in the dataset with the largest number of POIs, APE's token count reaches 67.7k, leading to an estimated API cost of $357.79 per 1000 queries. In comparison, **COMPASS** reduces this cost to $90.37 while maintaining high accuracy. The efficiency of **COMPASS** becomes even more pronounced in datasets with

Metric	Method	Toro	Melb	Edin	DisHolly	Epcot	CaliAdv	Disland	Average
Token Count	APE	5.4k	8.7k	11.6k	17.7k	25.5k	32.5k	67.7k	24.2k
	LLM-A*	2.8k	4.3k	**5.6k**	**8.5k**	**12.0k**	**15.3k**	**31.6k**	**11.4k**
	COMPASS	**2.6k**	**4.2k**	5.7k	8.6k	12.4k	15.8k	33.0k	11.8k
API Cost	APE	28.46	45.91	61.40	93.44	134.44	171.38	357.79	127.55
	LLM-A*	**3.55**	**5.49**	**7.14**	**10.79**	**15.14**	**19.24**	**39.69**	**14.43**
	COMPASS	7.19	11.59	15.49	23.58	33.91	43.25	90.37	32.20

Table 3: *Cost Comparison of Prompting Techniques: This table compares the total number of tokens used in the prompt & response and the estimated computational cost per 1000 queries to the GPT-4o API across various datasets. Techniques such as APE provide strong performance but come with significantly higher costs. LLM A*, on the other hand, is cheaper but has a lower performance. Whereas,* **COMPASS** *offers a more balanced approach, delivering competitive results without incurring the high computational overhead. Lower costs are generally more favorable for real-time applications.*

smaller POIs, where it incurs costs similar to LLM-A* while outperforming it in terms of retrieval quality.

This cost-effectiveness makes **COMPASS** highly suitable for large-scale applications where budget constraints and real-time processing requirements are critical. By reducing the dependency on high-context-window LLM calls, **COMPASS** provides a scalable and economically viable alternative to methods like APE while ensuring robust performance across different environments.

In summary, **COMPASS** demonstrates superior performance in path retrieval and generation while maintaining computational efficiency. Its ability to adapt without retraining, coupled with its lower operational costs, makes it a practical choice for real-world applications requiring efficient trajectory-based search and planning.

6 Conclusion and Future Work

In this work, we presented **COMPASS**, a framework utilizing large language models for spatial reasoning to solve the Popular Path Problem. Our experiments show that **COMPASS** is effective in generating paths under sparse data conditions and user-defined constraints. However, our results remain experimental, as **COMPASS** may generate suboptimal or invalid paths in certain cases, and the inherent stochasticity of LLMs poses challenges in consistency. Future work can focus on incorporating more contextual data, such as time and environmental factors, and enhancing the model's reliability through improved prompt engineering and dynamic memory management to handle larger datasets more effectively.

7 Limitations

While COMPASS demonstrates promising results in popular path discovery and synthesis, several limitations warrant consideration:

Large-scale Data Handling: COMPASS faces challenges when processing extensive datasets. The current implementation struggles with input sizes exceeding 128,000 tokens, due to the context window limitations of existing LLMs, which limits its applicability to very large or complex spatial networks. Moreover, according to Table 5, no further compression of **COMPASS** prompts are feasible.

Incompatibility with Internal Prompt Compression: Many LLMs employ internal prompt compression techniques to manage large inputs efficiently. However, **COMPASS**'s performance relies on the full, uncompressed prompt structure. Consequently, models that automatically compress prompts may not be compatible with our approach, potentially limiting the range of applicable LLMs.

Prompt and Reasoning Chain Optimization: While our current prompting strategy yields effective results, there may exist more optimized prompt structures or reasoning chains that could further enhance **COMPASS**'s performance. The current implementation, while effective, may not represent the absolute optimal prompt design for all scenarios.

References

1. Aghzal, M., Plaku, E., Yao, Z.: Can large language models be good path planners? a benchmark and investigation on spatial-temporal reasoning. arXiv preprint arXiv:2310.03249 (2023)
2. Banerjee, P., Ranu, S., Raghavan, S.: Inferring uncertain trajectories from partial observations. In: ICDM. pp. 30–39 (2014)
3. Chen, D., Ong, C., Xie, L.: Learning points and routes to recommend trajectories. In: CIKM 2016 - Proceedings of the 2016 ACM Conference on Information and Knowledge Management. pp. 2227–2232. International Conference on Information and Knowledge Management, Proceedings, Association for Computing Machinery (Oct 2016). https://doi.org/10.1145/2983323.2983672, publisher Copyright: © 2016 ACM.; 25th ACM International Conference on Information and Knowledge Management, CIKM 2016 ; Conference date: 24-10-2016 Through 28-10-2016
4. Chen, D., Ong, C.S., Xie, L.: Learning points and routes to recommend trajectories. In: The Conference on Information and Knowledge Management (CIKM). pp. 2227–2232 (2016)
5. Chen, Z., Shen, H.T., Zhou, X.: Discovering popular routes from trajectories. In: ICDE. pp. 900–911 (2011)
6. Chen, Z., Mao, H., Li, H., Jin, W., Wen, H., Wei, X., Wang, S., Yin, D., Fan, W., Liu, H., et al.: Exploring the potential of large language models (llms) in learning on graphs. ACM SIGKDD Explorations Newsletter **25**(2), 42–61 (2024)
7. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to algorithms. MIT press (2022)
8. Gundawar, A., Verma, M., Guan, L., Valmeekam, K., Bhambri, S., Kambhampati, S.: Robust planning with llm-modulo framework: Case study in travel planning. arXiv preprint arXiv:2405.20625 (2024)
9. Himoro, M.Y., Pareja-Lora, A.: Preliminary results on the evaluation of computational tools for the analysis of Quechua and Aymara. In: Proceedings of the Thirteenth Language Resources and Evaluation Conference. pp. 5450–5459. European Language Resources Association, Marseille, France (Jun 2022), https://aclanthology.org/2022.lrec-1.584

10. Jain, J., Bagadia, V., Manchanda, S., Ranu, S.: Neuromlr: Robust & reliable route recommendation on road networks. Advances in Neural Information Processing Systems **34**, 22070–22082 (2021)
11. Jang, M., Kwon, D.S., Lukasiewicz, T.: BECEL: Benchmark for consistency evaluation of language models. In: Proceedings of the 29th International Conference on Computational Linguistics. pp. 3680–3696. International Committee on Computational Linguistics, Gyeongju, Republic of Korea (Oct 2022), https://aclanthology.org/2022.coling-1.324
12. Kambhampati, S., Valmeekam, K., Guan, L., Stechly, K., Verma, M., Bhambri, S., Saldyt, L., Murthy, A.: Llms can't plan, but can help planning in llm-modulo frameworks. arXiv preprint arXiv:2402.01817 (2024)
13. Kruskal, J.B.: On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical society **7**(1), 48–50 (1956)
14. Laskar, M.T.R., Alqahtani, S., Bari, M.S., Rahman, M., Khan, M.A.M., Khan, H., Jahan, I., Bhuiyan, A., Tan, C.W., Parvez, M.R., et al.: A systematic survey and critical review on evaluating large language models: Challenges, limitations, and recommendations. arXiv preprint arXiv:2407.04069 (2024)
15. Li, X., Cong, G., Cheng, Y.: Spatial transition learning on road networks with deep probabilistic models. In: ICDE. pp. 349–360 (2020)
16. Li, Y., Dong, B., Lin, C., Guerin, F.: Compressing context to enhance inference efficiency of large language models. arXiv preprint arXiv:2310.06201 (2023)
17. Li, Z., Ding, B., Han, J., Kays, R.: Swarm: Mining relaxed temporal moving object clusters. Proceedings of the VLDB Endowment **3**(1-2), 723–734 (2010)
18. Lim, K.H., Chan, J., Leckie, C., Karunasekera, S.: Personalized trip recommendation for tourists based on user interests, points of interest visit durations and visit recency. Knowl. Inf. Syst. **54**(2), 375–406 (Feb 2018). https://doi.org/10.1007/s10115-017-1056-y, https://doi.org/10.1007/s10115-017-1056-y
19. Luo, W., Tan, H., Chen, L., Ni, L.M.: Finding time period-based most frequent path in big trajectory data. In: Proceedings of the 2013 ACM SIGMOD international conference on management of data. pp. 713–724 (2013)
20. Meng, S., Wang, Y., Yang, C.F., Peng, N., Chang, K.W.: Llm-a*: Large language model enhanced incremental heuristic search on path planning. arXiv preprint arXiv:2407.02511 (2024)
21. Meng, S., Wang, Y., Yang, C.F., Peng, N., Chang, K.W.: Llm-a*: Large language model enhanced incremental heuristic search on path planning (2024), https://arxiv.org/abs/2407.02511
22. Rashid, S.M.M., Ali, M.E., Cheema, M.A.: Deepalttrip: Top-k alternative itineraries for trip recommendation. IEEE Transactions on Knowledge and Data Engineering **35**(9), 9433–9447 (2023)
23. Robins, G., Zelikovsky, A.: Improved steiner tree approximation in graphs. In: SODA. pp. 770–779 (2000)
24. Schrijver, A., et al.: Combinatorial optimization: polyhedra and efficiency, vol. 24. Springer (2003)
25. Sharan, S., Pittaluga, F., Chandraker, M., et al.: Llm-assist: Enhancing closed-loop planning with language-based reasoning. arXiv preprint arXiv:2401.00125 (2023)
26. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning (2023), https://arxiv.org/abs/2303.11366
27. Song, C.H., Wu, J., Washington, C., Sadler, B.M., Chao, W.L., Su, Y.: Llm-planner: Few-shot grounded planning for embodied agents with large language models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 2998–3009 (2023)
28. Tam, Z.R., Wu, C.K., Tsai, Y.L., Lin, C.Y., Lee, H.y., Chen, Y.N.: Let me speak freely? a study on the impact of format restrictions on performance of large language models. arXiv preprint arXiv:2408.02442 (2024)

29. Wang, H., Feng, S., He, T., Tan, Z., Han, X., Tsvetkov, Y.: Can language models solve graph problems in natural language? Advances in Neural Information Processing Systems **36** (2024)
30. Wang, J., Wu, N., Zhao, W.X., Peng, F., Lin, X.: Empowering a* search algorithms with neural networks for personalized route recommendation. In: KDD. pp. 539–547 (2019)
31. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models (2023)
32. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models (2023)
33. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems **35**, 24824–24837 (2022)
34. Wei, L.Y., Zheng, Y., Peng, W.C.: Constructing popular routes from uncertain trajectories. In: SIGKDD. pp. 195–203 (2012)
35. Wu, H., Chen, Z., Sun, W., Zheng, B., Wang, W.: Modeling trajectories with recurrent neural networks. In: IJCAI. pp. 3083–3090 (2017)
36. Wu, H., Mao, J., Sun, W., Zheng, B., Zhang, H., Chen, Z., Wang, W.: Probabilistic robust route recovery with spatio-temporal dynamics. In: KDD. pp. 1915–1924 (2016)
37. Yang, C., Gidofalvi, G.: Fast map matching, an algorithm integrating hidden markov model with precomputation. International Journal of Geographical Information Science **32**(3), 547–570 (2018)
38. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601 (2023)
39. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: The Eleventh International Conference on Learning Representations (2023)
40. Ye, R., Zhang, C., Wang, R., Xu, S., Zhang, Y.: Natural language is all a graph needs. arXiv preprint arXiv:2308.07134 (2023)
41. Yuan, J., Zheng, Y., Xie, X., Sun, G.: Driving with knowledge from the physical world. In: Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining. pp. 316–324 (2011)
42. Yuan, J., Zheng, Y., Zhang, C., Xie, W., Xie, X., Sun, G., Huang, Y.: T-drive: driving directions based on taxi trajectories. In: Proceedings of the 18th SIGSPATIAL International conference on advances in geographic information systems. pp. 99–108 (2010)
43. Zheng, K., Zheng, Y., Xie, X., Zhou, X.: Reducing uncertainty of low-sampling-rate trajectories. In: ICDE. pp. 1144–1155 (2012)
44. Zheng, Y., Liu, Y., Yuan, J., Xie, X.: Urban computing with taxicabs. In: Proceedings of the 13th international conference on Ubiquitous computing. pp. 89–98 (2011)
45. Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J.: Large language models are human-level prompt engineers. arXiv preprint arXiv:2211.01910 (2022)

A COMPASS Algorithm

COMPASS is designed to identify a valid path between a given source and destination using historical trajectory data. The algorithm consists of several core functions that work together to query a language model, validate the results, and refine the response if necessary. In this section, we first describe the key functions used in the algorithm before presenting the complete workflow.

A.1 Function Descriptions

- `SEARCH`: Generates a structured prompt for the search phase and queries the language model (LLM). The function returns both the constructed prompt and the LLM-generated response.
- `ContextExtended`: When the initial response from the LLM is incomplete, this function re-queries the model with both the original prompt and the partial response to obtain a more complete answer.
- `GENERATE`: Similar to `SEARCH`, but used when no valid path is found. This function prompts the LLM to synthesize a new path by reasoning over the available trajectory data.
- `GraphGenerator`: Constructs a graph representation from historical trajectory data. This graph serves as a structural basis for validating generated paths.
- `isValid`: Verifies whether a generated path is reachable within the constructed graph. If the path does not exist in the historical data, further refinement is required.
- `SelfReflect`: Uses feedback from the `isValid` function to refine the LLM's output. It re-prompts the model with additional context to improve the accuracy and quality of the generated response.

A.2 Algorithm Overview

The full **COMPASS** algorithm is outlined in Algorithm 1. The process begins by attempting to find a valid path through the `SEARCH` function. If the response is incomplete, `ContextExtended` is invoked to refine it. If a valid path is not found, the algorithm proceeds to the `GENERATE` phase, attempting to synthesize a new path. The generated path is validated using `GraphGenerator` and `isValid`. If the path is deemed invalid, `SelfReflect` is used to iteratively refine the response.

B Synthetic Data Generation

To enhance the evaluative capacity of Large Language Models (LLMs) for spatial data through prompting techniques, we developed a method for creating synthetic trajectories that closely resemble real-world spatial patterns. Our approach encompasses a range of scenarios while maintaining control over the data generation process. Figure 6 illustrates the overall process. The following steps outline our synthetic data generation procedure:

Algorithm 1 COMPASS Algorithm

Require: $\mathcal{T}$ is a set of trajectories, s is the start point, d is the destination
Ensure: A valid path from s to d, or null if no valid path exists

```
prompt, response ← SEARCH(T, s, d)
if response is incomplete then
    response ← ContextExtended(prompt, response)
end if
if response.path is available then
    return response.path
end if
prompt, response ← GENERATE(T, s, d)
if response is incomplete then
    response ← ContextExtended(prompt, response)
end if
graph ← GraphGenerator(T)
if isValid(response.path, graph) then
    return response.path
else
    response ← SelfReflect(prompt, response)
end if
return response.path
```

1. **Graph Generation (G) [top-left]:** We initiate the process by formulating a graph G representing the spatial environment. This graph serves as the foundation for subsequent stages and is created using the "Reverse-delete algorithm" [13], generating a random connected graph with a specific number of edges.
2. **Node Selection (V_s):** We identify pivotal nodes within the generated graph, defining a set of important nodes V_s that are particularly relevant in the context of spatial data representation.
3. **Steiner Tree Construction (T_s) [top-right]:** Utilizing approximation algorithms, we construct a Steiner tree T_s by connecting the selected important nodes. This tree serves as a structural backbone for the synthetic spatial network.
4. **Highway Network Representation (H) [bottom-right]:** The Steiner tree is treated as a network of highways H, establishing an efficient and structured movement framework within the synthetic spatial environment.
5. **Trajectory Generation (τ) [bottom-left]:** We simulate realistic movement patterns by generating trajectories τ, which navigate from a source node to a destination node. This involves traversing the highway network and selecting optimal routes that closely mimic real-world scenarios. In cases where the efficient path is a direct route between two nodes, particularly when they are in close proximity, we opt for the simplest path rather than adhering strictly to the highway network.

Figure 7 shows that the synthetic data generated from the mentioned steps shows the patterns of real data. Table 4 provides a comparative analysis of our synthetic data generation efforts against real-world data, focusing on the generation of unique source-destination ground truth pairs. The "Result" column indicates the count of generated

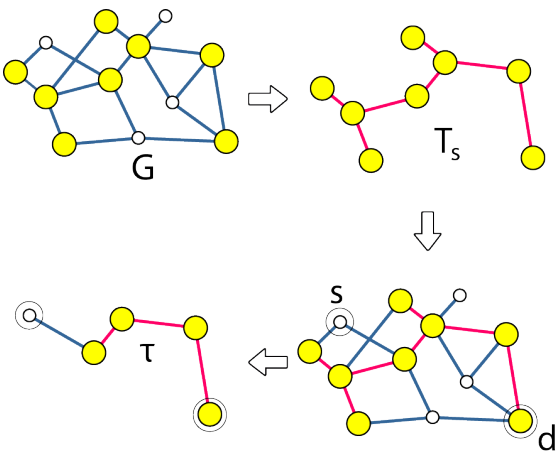


Fig. 6: *Synthetic Data Generation Process: Graph G represents our network structure, where yellow nodes denote pivotal nodes V_s. Using this information, we construct a Steiner tree T_s, which plays a crucial role in our trajectory generation process. Trajectories tend to align with the edges of the Steiner tree T_s, which serves as an efficient pathway resembling a 'highway' within our network.*

unique pairs, while the "Actual" column represents the corresponding count derived from actual data.

This methodology enables us to generate synthetic trajectories that closely model actual spatial data, providing a nuanced representation of spatial interactions for comprehensive evaluation of LLM performance in spatial data analysis.

C Additional Results & Trends

C.1 Varying Temperature

Fig 8 shows that LLMs generate better result with lower temperatures as a higher temperature can lead to hallucinating and a lower temperature gives out thoughtful answer. This led us to run all our experiments with temperature zero.

C.2 Varying Map Size

From Table 1, we see that results in larger datasets (more POIs & more trajectories) tends to have lesser F1 score. That is because it becomes less feasible to recommend a path that covers most of the popular POIs as the number of POIs increase. Having to keep a larger number of trajectories in context while inferring a popular path also adds to that factor. We can see this trend in Fig 5 as well where all of the figures have a downward trend.

Location	Node	Density	Routes	Result	Actual
Epcot	17	0.2	1248	212	207
Disneyland	31	0.4	2792	681	618
DisHolly	13	0.4	901	107	134

Table 4: *Comparison of synthetic and real-world data across multiple locations. The table presents key characteristics of the generated spatial datasets, including the number of nodes, spatial density, and the number of unique routes. The "Result" column denotes the number of unique source-destination pairs generated by our synthetic trajectory model, while the "Actual" column provides the corresponding real-world counts. A close match between the two indicates that our synthetic data effectively captures the structural patterns of real spatial trajectories.*

C.3 Consistency of Reachability

A higher reachability is always preferable but consistency is also necessary. Fig 9 shows that APE and **COMPASS** are the only high performing and consistence ones.

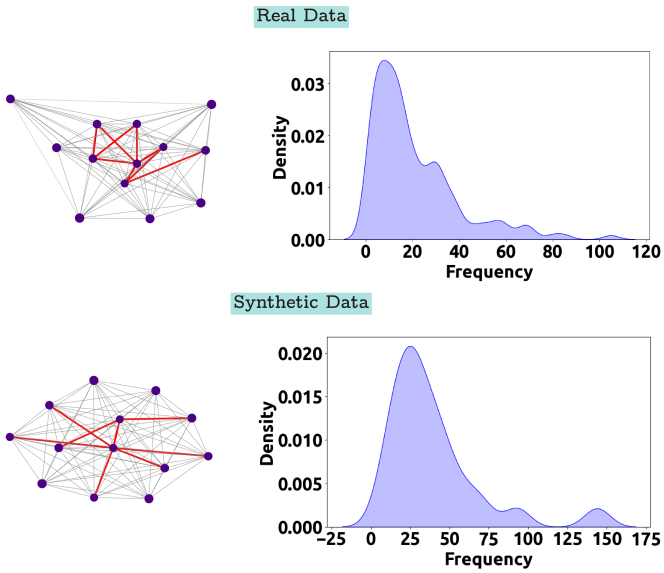


Fig. 7: *Comparison of Real and Synthetic Data: The synthetic data mimics the patterns of real data, validating the reliability of the generation process. The left diagram highlights the most frequently used subpaths in red, with less-traveled paths in grey, while the right diagram shows the frequency distribution of subpaths, offering insights into usage patterns across datasets. Here the vertical axis represents the density meaning the whole area under the curve equals to one.*

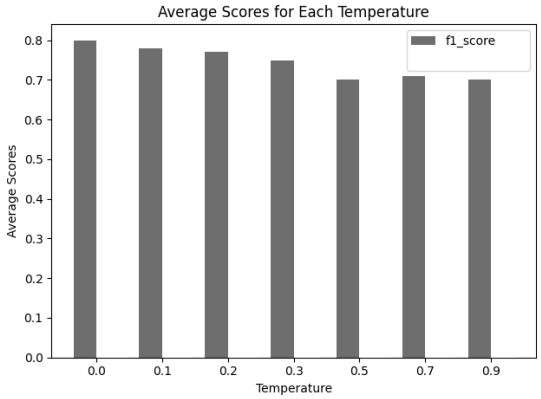


Fig. 8: *Average F1 Score across all dataset against varying temperature*

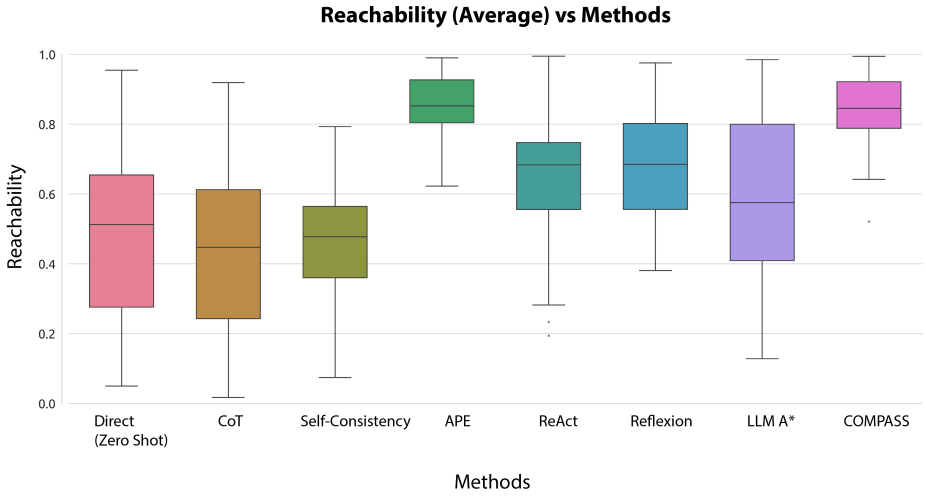


Fig. 9: *Consistency Across Prompting Techniques: This box plot illustrates the variation in reachability scores across different prompting techniques. Lower variance is preferable, as it indicates more consistent results.*

C.4 Reachability with Reduction

Selective Context [16] is a method that enhances the inference efficiency of LLMs by identifying and pruning redundancy in the input context to make the input more compact. Table 5 shows the comparison between **COMPASS**, Selective Context 10% and Selective Context 20% in average reduction and reachability across various datasets. This proves that **COMPASS** prompts are already compact enough themselves and no level of reduction can improve/maintain the same performance. It validates the low-cost of **COMPASS**.

Dataset	Metric	COMPASS Generate	Selective Context (10%)	Selective Context (20%)
Epcot	Reduction	0	16%	24%
	Reachability	0.84	0.15	0
CaliAdv	Reduction	0	14%	22%
	Reachability	0.91	0.12	0
Disland	Reduction	0	13%	20%
	Reachability	0.96	0.10	0.1
Average	Reduction	0	13.75%	21%
	Reachability	0.80	0.14	0.05

Table 5: *Comparison of Reduction and Reachability across various datasets and techniques. The table includes both reduction percentages and reachability scores for* **COMPASS** *Generate and two variations of Selective Context techniques (10% and 20%).*

Experiment	COMPASS Generate	Zero Shot	CoT	SC	ReAct	Reflexion	LLM A*	APE
Construct a path								
- with 4 nodes	8	0	5	6	7	8	7	1
- with 5 nodes	7	1	5	5	8	9	7	0
- that goes through x node	6	0	4	4	7	8	6	0
- that avoids x nodes	8	0	6	6	9	9	8	1

Table 6: *Results of path-generation experiments under various constraints (e.g., path length, inclusion/exclusion of specific nodes). Each cell shows the number of successful responses out of 10 trials for the listed method.*

C.5 Experiment with Various Constraints

COMPASS easily adapts to constraints using prompts, outperforming other models without requiring architectural changes. Table 6 summarizes its success in generating paths under three constraints: fixed-length paths, mandatory node inclusion, and node avoidance, demonstrating its flexibility and reliability.

D COMPASS Prompts

Two distinct prompts are utilized for the "Search" and "Generate" stages of **COMPASS**. The Search prompt instructs the Large Language Model (LLM) to identify the most popular path from a given set of trajectories, considering both path frequency and the popularity of individual nodes to optimize the F1 score. This approach ensures that the selected path not only reflects common routes but also incorporates highly frequented locations. In contrast, the Generate prompt is employed when no direct path exists

between the specified source and destination in the historical trajectory data. This prompt challenges the LLM to synthesize a new path by leveraging the available trajectory information. The LLM must analyze the existing paths, identify popular segments or nodes, and creatively combine them to construct a novel, yet likely popular, route connecting the given source and destination.

In the examples below, prompt 1 & 2.1 are Search prompts, the only difference being there is no Ground Truth in the latter query and LLM cannot find the popular path directly. So we go to prompt 2.2, where LLM is asked to curate a path. While the steps in each of the phases can be done in separate prompts, our test has shown that concatenating the steps of a each phase together in a single prompt yields the same results.

(1) Search Prompt

You are given a list of trajectories and a source-destination pair. Each trajectory is a list of numbers representing a path taken by someone, where each number corresponds to a specific location. Your task is to find the best path from the source to the destination that is not only the most popular but also includes the most popular POIs. Output a JSON with the keys:
Calculate Point of Interest Popularity (string): Step by step explain how the popularity of various points of interest (POIs) can be analyzed using historical trajectory data.
POI Popularity Ranking Analysis (string): Rank the POIs based on their frequency of visits or interactions in the dataset and provide the ranking.
Process of Identifying Existing Paths (string): Explain how we can identify existing paths from historical data.
Check Path Availability (string): Verify if there is a direct or indirect route between the source and destination within the dataset.
Extract All Paths Between 1 and 19 (string): Retrieve all possible routes connecting the source (s) and destination (d) from the historical data.
Evaluate Paths (string): Think step by step on how to evaluate paths and retrieve the best path.
Select Best Path (string): Explain how the evaluation can be used to select the best path.
Output the Best Path (string): Choose the most optimal route from the evaluated paths based on overall performance metrics.

Historical Trajectories:
10 -> 17 -> 28
1 -> 4 -> 8 -> 9 -> 10 -> 11 -> 15 -> 18 -> 19 -> 23 -> 24 -> 29
2 -> 3 -> 8 -> 9 -> 17 -> 29
1 -> 3 -> 17 -> 18 -> 29
. . .
2 -> 4 -> 24
4 -> 8 -> 15
1 -> 8 -> 15
Source: 1
Destination: 19

(1) Response

"Calculate Point of Interest Popularity": "To analyze the popularity of various points of interest (POIs) from historical trajectory data, we first examine how frequently each location is visited. By counting the number of times each location appears in different trajectories, we can infer its popularity. Points that are visited more often indicate higher significance or interest to the travelers. For example, if a location consistently appears across different paths, it can be flagged as a key point of interest.",
"POI Popularity Ranking Analysis": "We can rank the POIs by counting their occurrences in the historical trajectories. For example, locations like 9, 15, and 29 appear frequently, indicating that they are popular points of interest. A sample ranking based on frequency might look like this: 1. POI 9, 2. POI 15, 3. POI 29, 4. POI 8, and so on, with the ranking reflecting the number of occurrences of each location across all trajectories.",

(1) Response

"Process of Identifying Existing Paths": "To identify existing paths, we look for trajectories where both the source and destination locations are present. A path exists if the source appears before the destination within a trajectory. For instance, for the source 1 and destination 19, we can scan the trajectories to find all instances where 1 appears before 19.",
"Check Path Availability": "We can verify whether there is a direct or indirect route between the source and destination by checking all trajectories that contain both locations. A direct route exists if the source is immediately followed by the destination in a trajectory. Otherwise, an indirect route exists if there are intermediate locations between the source and destination. For example, the trajectory '1 -> 4 -> 9 -> 19' shows an indirect route between 1 and 19.",
"Extract All Paths Between 1 and 19": "All possible paths between source 1 and destination 19 are extracted by looking at the trajectories where 1 and 19 both appear. These paths may include different intermediate locations or be direct. For example, paths like '1 -> 4 -> 9 -> 19' or '1 -> 8 -> 9 -> 15 -> 19' can be extracted from the dataset.",
"Evaluate Paths": "To evaluate the paths, we consider multiple criteria such as the length of the path (number of steps), the frequency of visits to the intermediate locations, and how often the overall path is traveled. Paths with fewer steps and more popular POIs are often more desirable. For example, a path like '1 -> 9 -> 19' is more efficient compared to longer paths with more steps.",
"Select Best Path": "Once the paths are evaluated, the best path is selected based on performance metrics such as shortest path length and the popularity of the locations involved. For example, if the path '1 -> 9 -> 19' is both short and passes through a highly popular location (9), it might be considered the best path.",
"Output the Best Path": "The most optimal route based on the evaluation of available paths is: '1 -> 9 -> 19'. This path is both short and passes through the frequently visited location 9, making it the best choice."
}

(2.1) Search Prompt

You are given a list of historical trajectory and a source and a destination. Your goal is to find a popular path that goes through that source and destination. Output your response in json with following keys
Calculate Point of Interest Popularity (string): Step by step explain how the popularity of various points of interest (POIs) can be analyzing historical trajectory data
POI Popularity Ranking Analysis (string) : Rank the POIs based on their frequency of visits or interactions in the dataset and provide the ranking
Process of Identifying Existing Paths (string) : Explain how we can identify existing paths from historical data
Check Path Availability (string) : Verify if there is a direct or indirect route between the source and destination within the dataset.
Extract All Paths Between 1 and 21 (string) : Retrieve all possible routes connecting the source 1 and destination 21 from the historical data.
Evaluate Paths (string) : Think step by step on how to evaulate paths and retrieve the best path
Select Best Path (string) : Explain how the evaluation can be used to select the best path
Output the Best Path (string) : Choose the most optimal route from the evaluated paths based on overall performance metrics.

Historical Trajectories:
16 -> 9 -> 13
16 -> 4 -> 5 -> 10 -> 11 -> 2 -> 7 -> 15 -> 13
16 -> 2 -> 11 -> 9 -> 5 -> 8 -> 13
16 -> 5 -> 1 -> 9 -> 10 -> 17 -> 13
. . .
17 -> 10 -> 11 -> 4 -> 1 -> 16 -> 5 -> 15
17 -> 10 -> 6 -> 13 -> 15
1 -> 2 -> 7

Source: 1
Destination: 21

(2.1) Response

"Calculate Point of Interest Popularity": "To calculate the popularity of various points of interest (POIs), we analyze how frequently each location appears in the historical trajectories. The more times a point is visited across different trajectories, the higher its popularity. For instance, by parsing all the trajectories, we can count the occurrences of each POI to determine its popularity. This analysis can help highlight critical locations that people visit frequently, making them popular points of interest.",
"POI Popularity Ranking Analysis": "The POIs can be ranked based on the frequency of their appearance across the trajectories. For example, if POI 11 is visited 50 times and POI 5 is visited 30 times, POI 11 would rank higher. The ranking process involves counting the occurrences of each POI in all trajectories and sorting them in descending order to reflect their popularity. In this dataset, POIs such as 11, 5, and 8 are likely to rank highly based on their frequent appearances.",
"Process of Identifying Existing Paths": "To identify existing paths from historical data, we search for sequences of POIs in the given trajectories that match the source and destination points. By examining each trajectory, we look for segments that begin at the source (1) and end at the destination (21), considering both direct and indirect paths. The goal is to extract any paths that connect these two points directly or via other intermediate locations.",
"Check Path Availability": "To check if there is a direct or indirect route between the source (1) and destination (21), we scan the historical trajectories for any path that starts at 1 and ends at 21. If such a path exists, we note it as a valid route. If no direct path is found, we search for possible indirect routes that pass through intermediate POIs to connect the source and destination.",
. . .
}

(2.2) Generate Prompt

You are given a list of trajectories and a source-destination pair. Each trajectory is a list of numbers representing a path taken by someone. Each number is a specific location. For a given source and destination, there is no direct path in the graph. Your task is to generate the most popular path for the given source and destination. Output a JSON with the following keys:
Extract Possible Edges (string) : edges that can be found from the trajectory data
Analyze Edge Frequencies (string) : how to analyse edge frequency and find the most popular edges
Generate Path Candidates (string) : generate some possible path canditates from the above results
Score Path Candidates (string) : analytically generate some scores. Think step by step and provide them to the generated paths
Generate Reasoning for Path Selection (string) : think step by step on how to select a path
Output the Most Popular Path (string) : the most popular path

Historical Trajectories:
16 -> 9 -> 13
16 -> 4 -> 5 -> 10 -> 11 -> 2 -> 7 -> 15 -> 13
16 -> 2 -> 11 -> 9 -> 5 -> 8 -> 13
16 -> 5 -> 1 -> 9 -> 10 -> 17 -> 13
. . .
17 -> 10 -> 11 -> 4 -> 1 -> 16 -> 5 -> 15
17 -> 10 -> 6 -> 13 -> 15
1 -> 2 -> 7

Source: 1
Destination: 21

(2.2) Response

"Extract Possible Edges": "From the trajectory data, edges can be extracted as pairs of consecutive locations in each trajectory. For example, the first trajectory '10 -> 17 -> 28' gives edges (10, 17), (17, 28). Similarly, from '1 -> 4 -> 8 -> 9 -> 10 -> 11 -> 15 -> 18 -> 19 -> 23 -> 24 -> 29', edges include (1, 4), (4, 8), (8, 9), (9, 10), and so on. These form the possible edges between any two locations.",

(2.2) Response

"**Analyze Edge Frequencies**": "To analyze edge frequencies, count how often each edge appears across all trajectories. For example, edges like (1, 4) and (9, 15) occur multiple times, indicating they are commonly traveled. The most frequent edges should be prioritized for path generation.",
"**Generate Path Candidates**": "Based on the extracted edges and edge frequencies, possible path candidates from source 1 to destination 21 could be generated by identifying frequent paths that connect the source to the destination. Some possible paths include: (1 -> 4 -> 10 -> 19 -> 21), (1 -> 9 -> 15 -> 19 -> 21), and (1 -> 2 -> 9 -> 17 -> 19 -> 21).",
"**Score Path Candidates**": "To score the path candidates, consider factors such as the frequency of each edge along the path and the total number of edges. For example, (1 -> 9 -> 15 -> 19 -> 21) would score high if all the edges (1, 9), (9, 15), (15, 19), and (19, 21) are frequent. Another factor could be the total length of the path, with shorter paths preferred.",
"**Generate Reasoning for Path Selection**": "Paths that use more frequent edges are more likely to be popular. Additionally, paths with fewer total edges are generally more efficient and should be preferred. For example, (1 -> 9 -> 15 -> 19 -> 21) uses common edges and has a reasonable path length, making it a strong candidate.",
"**Output the Most Popular Path**": "(1 -> 9 -> 15 -> 19 -> 21) is selected as the most popular path as it uses highly frequent edges and is relatively short compared to other candidates."